

Name-Pranjal
USN- 1WA24CS212
Lab-1
FCFS
#include <stdio.h>

```
int main() {
    printf("Name-Pranjal, USN-1WA24CS212\n");

    int n,i;
    int pid[20],at[20],bt[20],ct[20],tat[20],wt[20];
    int completed[20]={0};
    int current_time=0,done=0;

    float avg_ct=0, avg_tat=0, avg_wt=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++){
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",i+1);
        scanf("%d%d",&at[i],&bt[i]);
    }

    while(done<n){

        int idx=-1;
        int min=9999;

        for(i=0;i<n;i++){
            if(at[i]<=current_time && completed[i]==0){
                if(bt[i]<min){
                    min=bt[i];
                    idx=i;
                }
            }
        }

        if(idx!=-1){
            current_time++;
        }
        else{
            ct[idx]=current_time+bt[idx];
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];

            current_time=ct[idx];
        }
    }
}
```

```

        completed[idx]=1;
        done++;
    }
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);

    avg_ct += ct[i];
    avg_tat += tat[i];
    avg_wt += wt[i];
}

avg_ct /= n;
avg_tat /= n;
avg_wt /= n;

printf("\nAverage Completion Time = %.2f", avg_ct);
printf("\nAverage Turnaround Time = %.2f", avg_tat);
printf("\nAverage Waiting Time = %.2f\n", avg_wt);

return 0;
}

```

OutPUT

```

Name-Pranjal, USN-1WA24CS212
Enter number of processes: 2
Enter AT and BT for P1: 0
1
Enter AT and BT for P2: 1
2

PID      AT      BT      CT      TAT      WT
P1       0       1       1       1       0
P2       1       2       3       2       0

Average Completion Time = 2.00
Average Turnaround Time = 1.50
Average Waiting Time = 0.00

Process returned 0 (0x0)   execution time : 32.109 s
Press any key to continue.

```

SJF-Non Primitive

```
#include <stdio.h>
```

```
int main() {  
    printf("Name-Pranjal, USN-1WA24CS212\n");
```

```
    int n,i;  
    int pid[20],at[20],bt[20],ct[20],tat[20],wt[20];  
    int completed[20]={0};  
    int current_time=0,done=0;
```

```
    float avg_ct=0, avg_tat=0, avg_wt=0;
```

```
    printf("Enter number of processes: ");  
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++){  
        pid[i]=i+1;  
        printf("Enter AT and BT for P%d: ",i+1);  
        scanf("%d%d",&at[i],&bt[i]);  
    }
```

```
    while(done<n){
```

```
        int idx=-1;  
        int min=9999;
```

```
        for(i=0;i<n;i++){  
            if(at[i]<=current_time && completed[i]==0){  
                if(bt[i]<min){  
                    min=bt[i];  
                    idx=i;  
                }  
            }  
        }
```

```
        if(idx== -1){  
            current_time++;  
        }
```

```
        else{  
            ct[idx]=current_time+bt[idx];  
            tat[idx]=ct[idx]-at[idx];  
            wt[idx]=tat[idx]-bt[idx];
```

```
            current_time=ct[idx];  
            completed[idx]=1;  
            done++;  
        }
```

```

}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);

    avg_ct += ct[i];
    avg_tat += tat[i];
    avg_wt += wt[i];
}

avg_ct /= n;
avg_tat /= n;
avg_wt /= n;

printf("\nAverage Completion Time = %.2f", avg_ct);
printf("\nAverage Turnaround Time = %.2f", avg_tat);
printf("\nAverage Waiting Time = %.2f\n", avg_wt);

return 0;
}

```

Output

```

Name-Pranjal, USN-1WA24CS212
Enter number of processes: 3
Enter AT and BT for P1: 0
1
Enter AT and BT for P2: 1
2
Enter AT and BT for P3: 2
3

PID    AT    BT    CT    TAT    WT
P1      0     1     1     1     0
P2      1     2     3     2     0
P3      2     3     6     4     1

Average Completion Time = 3.33
Average Turnaround Time = 2.33
Average Waiting Time = 0.33

Process returned 0 (0x0)  execution time : 74.224 s
Press any key to continue.
|

```

SJF Primitive

```
#include <stdio.h>
```

```

int main(){

    int n,i;
    int pid[20],at[20],bt[20],rt[20];
    int ct[20],tat[20],wt[20];

    float avg_ct=0, avg_tat=0, avg_wt=0;

    printf("Name-Pranjal, USN-1WA24CS212\n");

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++){
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",i+1);
        scanf("%d%d",&at[i],&bt[i]);
        rt[i]=bt[i];
    }
}

```

```
int completed=0,current_time=0;
```

```
while(completed<n){
```

```
    int idx=-1;
```

```
    int min=9999;
```

```
    for(i=0;i<n;i++){
```

```
        if(at[i]<=current_time && rt[i]>0){
```

```
            if(rt[i]<min){
```

```
                min=rt[i];
```

```
                idx=i;
```

```
            }
```

```
        }
```

```
    }
```

```
    if(idx==1){
```

```
        current_time++;
```

```
    }
```

```
    else{
```

```
        rt[idx]--;
```

```
        current_time++;
```

```
        if(rt[idx]==0){
```

```
            ct[idx]=current_time;
```

```
            tat[idx]=ct[idx]-at[idx];
```

```
            wt[idx]=tat[idx]-bt[idx];
```

```
            completed++;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
for(i=0;i<n;i++){
```

```
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
```

```
    avg_ct += ct[i];
```

```
    avg_tat += tat[i];
```

```
    avg_wt += wt[i];
```

```
}
```

```
avg_ct /= n;
```

```
avg_tat /= n;
```

```
avg_wt /= n;
```

```
printf("\nAverage Completion Time = %.2f", avg_ct);
```

```

printf("\nAverage Turnaround Time = %.2f", avg_tat);
printf("\nAverage Waiting Time = %.2f\n", avg_wt);

return 0;
}

```

```

Name-Pranjal, USN-1WA24CS212
Enter number of processes: 3
Enter AT and BT for P1: 0
1
Enter AT and BT for P2: 1
2
Enter AT and BT for P3: 2
3

PID    AT    BT    CT    TAT    WT
P1      0     1     1     1     0
P2      1     2     3     2     0
P3      2     3     6     4     1

Average Completion Time = 3.33
Average Turnaround Time = 2.33
Average Waiting Time = 0.33

Process returned 0 (0x0)   execution time : 74.224 s
Press any key to continue.

```

PriorityScheduling Primitive

```
#include <stdio.h>
```

```

int main()
{
    int n;
    int at[10], bt[10], ct[10], tat[10], wt[10], rt[10];
    int i;
    printf("Name-Pranjal, USN-1wa24CS212");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    // input
    for (i = 0; i < n; i++)
    {
        printf("Enter AT and BT for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }
}

```

```

// completion time
for (i = 0; i < n; i++)
{
    if (i == 0)
        ct[i] = at[i] + bt[i];
    else
    {
        if (ct[i - 1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i - 1] + bt[i];
    }
}

// tat wt rt
for (i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    rt[i] = wt[i];
}

// table
printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}

// formulas
printf("\nFormulas:\n");
printf("CT = Previous CT + BT\n");
printf("TAT = CT - AT\n");
printf("WT = TAT - BT\n");
printf("RT = WT\n");

return 0;
}

```

```

Name-Pranjal, USN-1wa24CS212Enter number of processes: 2
Enter AT and BT for P1: 0
1
Enter AT and BT for P2: 2
3

P      AT      BT      CT      TAT      WT      RT
P1      0      1      1      1      0      0
P2      2      3      5      3      0      0

Formulas:
CT = Previous CT + BT
TAT = CT - AT
WT = TAT - BT
RT = WT

Process returned 0 (0x0)  execution time : 16.845 s
Press any key to continue.
|

```

Priority Sheduling Non Primitive

```
#include <stdio.h>
```

```

int main()
{
    int n;
    int at[10], bt[10], ct[10], tat[10], wt[10];
    int i, sum = 0;
    printf("Name-Pranjal, USN-1wa24CS212");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Input
    for (i = 0; i < n; i++)
    {
        printf("Enter Arrival Time and Burst Time of P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    // Completion Time
    for (i = 0; i < n; i++)
    {
        if (i == 0)
            ct[i] = at[i] + bt[i];
        else

```

```

    {
        if (ct[i - 1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i - 1] + bt[i];
    }
}

// Turnaround Time and Waiting Time
for (i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i]; // TAT formula
    wt[i] = tat[i] - bt[i]; // WT formula
}

// Table
printf("\n-----\n");
printf("P\tAT\tBT\tCT\tTAT\tWT\n");
printf("-----\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("-----\n");

// Formula table
printf("\nFormulas used:\n");

printf("Completion Time (CT) = Previous CT + BT\n");
printf("Turnaround Time (TAT) = CT - AT\n");
printf("Waiting Time (WT) = TAT - BT\n");
printf("Runtime = Burst Time\n");

return 0;
}

```

```

Name-Pranjal, USN-1wa24CS212Enter number of processes: 3
Enter Arrival Time and Burst Time of P1: 1
2
Enter Arrival Time and Burst Time of P2: 3
4
Enter Arrival Time and Burst Time of P3: 5
6

```

P	AT	BT	CT	TAT	WT
P1	1	2	3	2	0
P2	3	4	7	4	0
P3	5	6	13	8	2

Formulas used:

Completion Time (CT) = Previous CT + BT

Turnaround Time (TAT) = CT - AT

Waiting Time (WT) = TAT - BT

Runtime = Burst Time

Process returned 0 (0x0) execution time : 14.190 s
Press any key to continue.

Round Robin Primitive

```
#include <stdio.h>
```

```

int main()
{
    int n, tq;
    int bt[10], rt[10], ct[10], tat[10], wt[10], at[10];
    int i, time = 0, remain, flag = 0;
    printf("Name-Pranjal, USN-1wa24CS212");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    remain = n;

    for (i = 0; i < n; i++)
    {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
}

```

```

    rt[i] = bt[i];
    at[i] = 0;
}

printf("Enter Time Quantum: ");
scanf("%d", &tq);

i = 0;

while (remain != 0)
{
    if (rt[i] <= tq && rt[i] > 0)
    {
        time += rt[i];
        rt[i] = 0;
        flag = 1;
    }
    else if (rt[i] > 0)
    {
        rt[i] -= tq;
        time += tq;
    }

    if (rt[i] == 0 && flag == 1)
    {
        remain--;
        ct[i] = time;
        flag = 0;
    }

    if (i == n - 1)
        i = 0;
    else
        i++;
}

for (i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
}

printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i], tat[i], wt[i], bt[i]);
}

```

```

}

printf("\nFormulas:\n");
printf("CT = finishing time\n");
printf("TAT = CT - AT\n");
printf("WT = TAT - BT\n");
printf("RT = Burst Time\n");

return 0;
}

```

Output

```

Name-Pranjal, USN-1wa24CS212Enter number of processes: 3
Enter Burst Time for P1: 6
Enter Burst Time for P2: 7
Enter Burst Time for P3: 1
Enter Time Quantum: 2

P      AT      BT      CT      TAT      WT      RT
P1      0       6      11      11       5       6
P2      0       7      14      14       7       7
P3      0       1       5       5        4       1

Formulas:
CT = finishing time
TAT = CT - AT
WT = TAT - BT
RT = Burst Time

Process returned 0 (0x0)   execution time : 9.882 s
Press any key to continue.
|

```

Round Robin Non-primitive

```
#include <stdio.h>
```

```

void roundRobin(int n, int bt[], int tq,
               int ct[], int tat[], int wt[], int at[])
{
    int rt[10];
    int time = 0, remain, i, flag = 0;

    remain = n;

    for (i = 0; i < n; i++)
        rt[i] = bt[i];
}

```

```

i = 0;

while (remain != 0)
{
    if (rt[i] <= tq && rt[i] > 0)
    {
        time += rt[i];
        rt[i] = 0;
        flag = 1;
    }
    else if (rt[i] > 0)
    {
        rt[i] -= tq;
        time += tq;
    }

    if (rt[i] == 0 && flag == 1)
    {
        remain--;
        ct[i] = time;
        flag = 0;
    }

    if (i == n - 1)
        i = 0;
    else
        i++;
}

for (i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
}
}

int main()
{
    int n, tq;
    int bt[10], ct[10], tat[10], wt[10], at[10];
    int i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {

```

```

        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        at[i] = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    roundRobin(n, bt, tq, ct, tat, wt, at);

    printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for (i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            i + 1, at[i], bt[i], ct[i], tat[i], wt[i], bt[i]);
    }

    printf("\nFormulas:\n");
    printf("CT = finishing time\n");
    printf("TAT = CT - AT\n");
    printf("WT = TAT - BT\n");
    printf("RT = Burst Time\n");

    return 0;
}

```

```

Enter number of processes: 3
Enter Burst Time for P1: 2
Enter Burst Time for P2: 3
Enter Burst Time for P3: 4
Enter Time Quantum: 2

```

P	AT	BT	CT	TAT	WT	RT
P1	0	2	2	2	0	2
P2	0	3	7	7	4	3
P3	0	4	9	9	5	4

Formulas:

CT = finishing time

TAT = CT - AT

WT = TAT - BT

RT = Burst Time

Process returned 0 (0x0) execution time : 14.286 s

Press any key to continue.

|

Multilevel Queue

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i;
```

```
    int at[20], bt[20], ct[20], tat[20], wt[20];
```

```
    int q[20];
```

```
    int rt[20];
```

```
    int tq;
```

```
    int time = 0;
```

```
    int done;
```

```
    float avgCT = 0, avgTAT = 0, avgWT = 0;
```

```
    printf("Name-Pranjal, USN-1wa24CS212");
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    // input
```

```

for (i = 0; i < n; i++)
{
    printf("\nProcess P%d\n", i + 1);

    printf("Enter Queue number (1 or 2): ");
    scanf("%d", &q[i]);

    printf("Enter Arrival Time: ");
    scanf("%d", &at[i]);

    printf("Enter Burst Time: ");
    scanf("%d", &bt[i]);

    rt[i] = bt[i];
}

printf("\nEnter Time Quantum for Queue 1 (RR): ");
scanf("%d", &tq);

// ----- Queue 1 Round Robin -----
while (1)
{
    done = 1;

    for (i = 0; i < n; i++)
    {
        if (q[i] == 1 && rt[i] > 0 && at[i] <= time)
        {
            done = 0;

            if (rt[i] > tq)
            {
                time += tq;
                rt[i] -= tq;
            }
            else
            {
                time += rt[i];
                rt[i] = 0;
                ct[i] = time;
            }
        }
    }
}

if (done)
    break;
}

```

```

// ----- Queue 2 FCFS -----
for (i = 0; i < n; i++)
{
    if (q[i] == 2)
    {
        if (time < at[i])
            time = at[i];

        time += bt[i];
        ct[i] = time;
    }
}

// ----- TAT WT -----
for (i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    avgCT += ct[i];
    avgTAT += tat[i];
    avgWT += wt[i];
}

avgCT /= n;
avgTAT /= n;
avgWT /= n;

// ----- Table -----
printf("\n-----\n");
printf("P\tQ\tAT\tBT\tCT\tTAT\tWT\n");
printf("-----\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, q[i], at[i], bt[i],
        ct[i], tat[i], wt[i]);
}

printf("-----\n");

printf("\nAverage CT = %.2f", avgCT);
printf("\nAverage TAT = %.2f", avgTAT);
printf("\nAverage WT = %.2f\n", avgWT);

printf("\nFormulas:\n");
printf("CT = finishing time\n");

```

```

printf("TAT = CT - AT\n");
printf("WT = TAT - BT\n");

return 0;
}

```

Name-Pranjal, USN-1wa24CS212Enter number of processes: 3

Process P1

Enter Queue number (1 or 2): 2

Enter Arrival Time: 0

Enter Burst Time: 1

Process P2

Enter Queue number (1 or 2): 3

Enter Arrival Time: 1

Enter Burst Time: 0

Process P3

Enter Queue number (1 or 2): 4

Enter Arrival Time: 5

Enter Burst Time: 2

Enter Time Quantum for Queue 1 (RR): 2

P	Q	AT	BT	CT	TAT	WT
P1	2	0	1	1	1	0
P2	3	1	0	13	12	12
P3	4	5	2	7864944	7864939	7864937

Average CT = 2621652.75

Average TAT = 2621650.75

Average WT = 2621649.75

Formulas:

CT = finishing time

TAT = CT - AT

WT = TAT - BT

Process returned 0 (0x0) execution time : 23.387 s

Press any key to continue.

|

```

RMS.c
#include <stdio.h>
#include <math.h>

// GCD
int gcd(int a, int b) {
    while(b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// LCM
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

typedef struct {
    int id;
    int C, T;
    int remaining;
    int next_arrival;
} Task;

int main() {
    int n;
    float U = 0, Ubound;
    printf("Name-Pranjal, USN-1wa24CS212");

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    Task t[n];

    // Input
    for(int i = 0; i < n; i++) {
        printf("\nTask %d\n", i + 1);
        t[i].id = i + 1;

        printf("Enter Computation Time (Ci): ");
        scanf("%d", &t[i].C);

        printf("Enter Period (Ti): ");
        scanf("%d", &t[i].T);

        t[i].remaining = 0;
    }
}

```

```

    t[i].next_arrival = 0;

    U += (float)t[i].C / t[i].T;
}

// Calculate Ubound
Ubound = n * (pow(2, (float)1/n) - 1);

printf("\nU = %.3f", U);
printf("\nUbound = %.3f\n", Ubound);

// ● CONDITION CHECK
if(U > Ubound) {
    printf("\nSystem is NOT schedulable using RMS\n");
    return 0;
}

printf("\nSystem is SCHEDULABLE → Printing Gantt Chart...\n");

// ♦ Find LCM (hyperperiod)
int hyper = t[0].T;
for(int i = 1; i < n; i++) {
    hyper = lcm(hyper, t[i].T);
}

int timeline[200];

// ♦ RMS Scheduling
for(int time = 0; time < hyper; time++) {

    // arrivals
    for(int i = 0; i < n; i++) {
        if(time == t[i].next_arrival) {
            t[i].remaining = t[i].C;
            t[i].next_arrival += t[i].T;
        }
    }

    // select smallest period
    int idx = -1, min_period = 9999;

    for(int i = 0; i < n; i++) {
        if(t[i].remaining > 0 && t[i].T < min_period) {
            min_period = t[i].T;
            idx = i;
        }
    }
}

```

```

        if(idx != -1) {
            timeline[time] = t[idx].id;
            t[idx].remaining--;
        } else {
            timeline[time] = -1;
        }
    }
}

// ♦ PRINT CHART
printf("\nGantt Chart:\n");

// tasks line
for(int i = 0; i < hyper; i++) {
    if(timeline[i] == -1)
        printf("- ");
    else
        printf("T%d ", timeline[i]);
}

// time line
printf("\n");
for(int i = 0; i < hyper; i++) {
    printf("%d ", i);
}

printf("\n");

return 0;
}

```

```

Name-Pranjal, USN-lwa24CS212Enter number of tasks: 2
Task 1
Enter Computation Time (Ci): 1
Enter Period (Ti): 100
Task 2
Enter Computation Time (Ci): 2
Enter Period (Ti): 10
U = 0.210
Ubound = 0.828
System is SCHEDULABLE f&E Printing Gantt Chart...
Gantt Chart:
T2 T2 T1 - - - - - T2 T2 - - - - - T2 T2 - - - - - T2 T2 - - - - - T2 T2 - - - - - T2 T2 - - - - - T2 T2 -
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43
44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85
86 87 88 89 90 91 92 93 94 95 96 97 98 99
Process returned 0 (0x0) execution time : 16.788 s
Press any key to continue.

```

Earliest Deadline First

```

#include <stdio.h>
#include <math.h>

```

```

// GCD
int gcd(int a, int b) {

```

```

while(b != 0) {
    int temp = b;
    b = a % b;
    a = temp;
}
return a;
}

// LCM
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

typedef struct {
    int id;
    int C, T;
    int remaining;
    int next_arrival;
} Task;

int main() {
    int n;
    float U = 0, Ubound;
    printf("Name-Pranjal, USN-1wa24CS212");

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    Task t[n];

    // Input
    for(int i = 0; i < n; i++) {
        printf("\nTask %d\n", i + 1);
        t[i].id = i + 1;

        printf("Enter Computation Time (Ci): ");
        scanf("%d", &t[i].C);

        printf("Enter Period (Ti): ");
        scanf("%d", &t[i].T);

        t[i].remaining = 0;
        t[i].next_arrival = 0;

        U += (float)t[i].C / t[i].T;
    }

    // Calculate Ubound

```

```

Ubound = n * (pow(2, (float)1/n) - 1);

printf("\nU = %.3f", U);
printf("\nUbound = %.3f\n", Ubound);

// ● CONDITION CHECK
if(U > Ubound) {
    printf("\nSystem is NOT schedulable using RMS\n");
    return 0;
}

printf("\nSystem is SCHEDULABLE → Printing Gantt Chart...\n");

// ♦ Find LCM (hyperperiod)
int hyper = t[0].T;
for(int i = 1; i < n; i++) {
    hyper = lcm(hyper, t[i].T);
}

int timeline[200];

// ♦ RMS Scheduling
for(int time = 0; time < hyper; time++) {

    // arrivals
    for(int i = 0; i < n; i++) {
        if(time == t[i].next_arrival) {
            t[i].remaining = t[i].C;
            t[i].next_arrival += t[i].T;
        }
    }

    // select smallest period
    int idx = -1, min_period = 9999;

    for(int i = 0; i < n; i++) {
        if(t[i].remaining > 0 && t[i].T < min_period) {
            min_period = t[i].T;
            idx = i;
        }
    }

    if(idx != -1) {
        timeline[time] = t[idx].id;
        t[idx].remaining--;
    } else {
        timeline[time] = -1;
    }
}

```

```

}

// ♦ PRINT CHART
printf("\nGantt Chart:\n");

// tasks line
for(int i = 0; i < hyper; i++) {
    if(timeline[i] == -1)
        printf("- ");
    else
        printf("T%d ", timeline[i]);
}

// time line
printf("\n");
for(int i = 0; i < hyper; i++) {
    printf("%d ", i);
}

printf("\n");

return 0;
}

```

```

Name-Pranjal, USN-1wa24CS212Enter number of tasks: 2

Task 1
Enter Computation Time (Ci): 1
Enter Period (Ti): 10

Task 2
Enter Computation Time (Ci): 2
Enter Period (Ti): 15

U = 0.233
Ubound = 0.828

System is SCHEDULABLE Printing Gantt Chart...

Gantt Chart:
T1 T2 T2 - - - - - T1 - - - - T2 T2 - - - T1 - - - - -
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29

Process returned 0 (0x0) execution time : 20.451 s
Press any key to continue.
|

```

Proportional

```
#include <stdio.h>
```

```

int main() {
    int n, i;
    printf("Name-Pranjal, USN-1wa24CS212");

    printf("Enter number of processes: ");
    scanf("%d", &n);
}

```

```

int bt[n], share[n];
float cpu[n];

int totalShare = 0, totalBurst = 0;

// Input burst time and share
for(i = 0; i < n; i++) {
    printf("\nProcess P%d\n", i+1);

    printf("Enter Burst Time: ");
    scanf("%d", &bt[i]);

    printf("Enter Share (Tickets): ");
    scanf("%d", &share[i]);

    totalShare += share[i];
    totalBurst += bt[i];
}

// Calculate proportional CPU time
for(i = 0; i < n; i++) {
    cpu[i] = ((float)share[i] / totalShare) * totalBurst;
}

// Display result
printf("\n--- Proportional Share Scheduling ---\n");
printf("Process\tBurst Time\tShare\tAllocated CPU Time\n");

for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%.2f\n", i+1, bt[i], share[i], cpu[i]);
}

return 0;
}

```

Name-Pranjal, USN-1wa24CS212Enter number of processes: 3

Process P1

Enter Burst Time: 1

Enter Share (Tickets): 11

Process P2

Enter Burst Time: 2

Enter Share (Tickets): 13

Process P3

Enter Burst Time: 3

Enter Share (Tickets): 14

--- Proportional Share Scheduling ---

Process	Burst Time	Share	Allocated CPU Time
P1	1	11	1.74
P2	2	13	2.05
P3	3	14	2.21

Process returned 0 (0x0) execution time : 18.684 s

Press any key to continue.

Producer-consumer Problem using semaphore

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <windows.h>
```

```
#define MAX 100
```

```
int buffer[MAX];
```

```
int in = 0, out = 0;
```

```
int n;
```

```
HANDLE empty, full, mutex;
```

```
// Producer
```

```
void produce(int item) {
```

```
    if (WaitForSingleObject(empty, 0) != WAIT_OBJECT_0) {
```

```
        printf("Buffer is FULL! Cannot produce.\n");
```

```
        return;
```

```
    }
```

```
    WaitForSingleObject(mutex, INFINITE);
```

```

    buffer[in] = item;
    printf("Produced: %d at position %d\n", item, in);
    in = (in + 1) % n;

    ReleaseMutex(mutex);
    ReleaseSemaphore(full, 1, NULL);
}

// Consumer
void consume() {
    int item;

    if (WaitForSingleObject(full, 0) != WAIT_OBJECT_0) {
        printf("Buffer is EMPTY! Cannot consume.\n");
        return;
    }

    WaitForSingleObject(mutex, INFINITE);

    item = buffer[out];
    printf("Consumed: %d from position %d\n", item, out);
    out = (out + 1) % n;

    ReleaseMutex(mutex);
    ReleaseSemaphore(empty, 1, NULL);
}

int main() {
    int choice, item;
    printf("Name-Pranjal, USN-1wa24CS212");

    printf("Enter buffer size: ");
    scanf("%d", &n);

    empty = CreateSemaphore(NULL, n, n, NULL);
    full = CreateSemaphore(NULL, 0, n, NULL);
    mutex = CreateMutex(NULL, FALSE, NULL);

    while (1) {
        printf("\n--- MENU ---\n");
        printf("1. Producer\n");
        printf("2. Consumer\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:

```

```
    printf("Enter item to produce: ");
    scanf("%d", &item);
    produce(item);
    break;

case 2:
    consume();
    break;

case 3:
    printf("Exiting...\n");

    CloseHandle(empty);
    CloseHandle(full);
    CloseHandle(mutex);
    return 0;

default:
    printf("Invalid choice!\n");
}
}
}
```

Name-Pranjal, USN-1wa24CS212Enter buffer size: 5

--- MENU ---

1. Producer

2. Consumer

3. Exit

Enter your choice: 1

Enter item to produce: 2

Produced: 2 at position 0

--- MENU ---

1. Producer

2. Consumer

3. Exit

Enter your choice: 1

Enter item to produce: 2

Produced: 2 at position 1

--- MENU ---

1. Producer

2. Consumer

3. Exit

Enter your choice: 1

Enter item to produce: 2

Produced: 2 at position 2

--- MENU ---

1. Producer

2. Consumer

3. Exit

Enter your choice: 8

Invalid choice!

--- MENU ---

1. Producer

2. Consumer

3. Exit

Enter your choice: 9

Invalid choice!

--- MENU ---

1. Producer

2. Consumer

3. Exit

Enter your choice: 7

Invalid choice!

--- MENU ---

1. Producer

2. Consumer

3. Exit

Dining Philispher

```
/*
 * Dining Philosophers Problem
 * Unit 3 - Process Synchronization (Chapter 5)
 *
 * PSEUDOCODE FROM NOTES (structure of Philosopher i):
 * -----
 * do {
 *     wait (chopstick[i]);      <- pick up LEFT chopstick
 *     wait (chopstick[(i+1) % 5]);  <- pick up RIGHT chopstick
 *     // eat
 *     signal (chopstick[i]);      <- put down LEFT chopstick
 *     signal (chopstick[(i+1) % 5]);  <- put down RIGHT chopstick
 *     // think
 * } while (TRUE);
 *
 * PROBLEM with above: if ALL philosophers pick up left chopstick
 * simultaneously -> DEADLOCK.
 *
 * DEADLOCK SOLUTION FROM NOTES:
 * Allow at most (N-1) philosophers to sit at a time.
 * -> This guarantees at least one philosopher can always eat.
 *
 * Shared data:
 * Semaphore chopstick[N] initialized to 1 (1 = available)
 * Here semaphore is simulated as integer: wait() = -1, signal() = +1
 */
```

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int chopstick[MAX]; /* semaphore: 1 = available, 0 = taken */
int sitting;        /* how many philosophers are currently sitting */
int n;              /* total philosophers */
```

```
/* wait(chopstick[i]) — tries to pick up chopstick */
```

```
int wait_chopstick(int i) {
    if (chopstick[i] == 1) {
        chopstick[i] = 0; /* acquire */
        return 1;         /* success */
    }
    return 0;              /* fail — chopstick busy */
}
```

```
/* signal(chopstick[i]) — puts down chopstick */
```

```
void signal_chopstick(int i) {
    chopstick[i] = 1; /* release */
}
```

```

}

void show_table() {
    int i;
    printf("\n Chopsticks: ");
    for (i = 0; i < n; i++)
        printf("[C%d:%s] ", i, chopstick[i] ? "FREE" : "BUSY");
    printf("\n Philosophers sitting at table: %d / %d (max allowed: %d)\n",
        sitting, n, n - 1);
}

void philosopher_eat(int i) {
    int left = i;
    int right = (i + 1) % n;

    printf("\n--- Philosopher %d wants to EAT ---\n", i);

    /* DEADLOCK PREVENTION: allow at most (n-1) philosophers to sit */
    if (sitting >= n - 1) {
        printf(" Table full! Max %d philosophers allowed to sit.\n", n - 1);
        printf(" Philosopher %d must WAIT (deadlock prevention).\n", i);
        return;
    }

    sitting++;
    printf(" Philosopher %d sits down. (%d sitting now)\n", i, sitting);

    /* wait(chopstick[i]) — pick up LEFT */
    if (!wait_chopstick(left)) {
        printf(" Chopstick %d (left) is BUSY. Philosopher %d must WAIT.\n", left, i);
        sitting--;
        return;
    }
    printf(" wait(chopstick[%d]) — Philosopher %d picks up LEFT chopstick.\n", left, i);

    /* wait(chopstick[(i+1)%n]) — pick up RIGHT */
    if (!wait_chopstick(right)) {
        printf(" Chopstick %d (right) is BUSY. Philosopher %d must WAIT.\n", right, i);
        signal_chopstick(left); /* put left back */
        sitting--;
        return;
    }
    printf(" wait(chopstick[%d]) — Philosopher %d picks up RIGHT chopstick.\n", right, i);

    /* EAT */
    printf(" >>> Philosopher %d is EATING <<<\n", i);

    /* signal(chopstick[i]) — put down LEFT */

```

```

signal_chopstick(left);
printf(" signal(chopstick[%d]) — Philosopher %d puts down LEFT chopstick.\n", left, i);

/* signal(chopstick[(i+1)%n]) — put down RIGHT */
signal_chopstick(right);
printf(" signal(chopstick[%d]) — Philosopher %d puts down RIGHT chopstick.\n", right, i);

/* THINK */
printf(" Philosopher %d is now THINKING.\n", i);
sitting--;
}

int main() {
    int i, choice, p;

    printf("=====\n");
    printf(" Dining Philosophers Problem\n");
    printf("=====\n");
    printf("Enter number of philosophers (2-%d): ", MAX);
    scanf("%d", &n);

    if (n < 2 || n > MAX) {
        printf("Invalid! Enter between 2 and %d.\n", MAX);
        return 1;
    }

    /* Initialize: all chopsticks available (semaphore = 1) */
    for (i = 0; i < n; i++)
        chopstick[i] = 1;
    sitting = 0;

    printf("\nSetup: %d Philosophers, %d Chopsticks (circular table)\n", n, n);
    printf("Deadlock rule: At most %d philosophers can sit at once.\n", n - 1);

    /* Simulation loop */
    while (1) {
        show_table();
        printf("\n1. Philosopher eats\n2. Exit\nChoice: ");
        scanf("%d", &choice);

        if (choice == 2) break;

        if (choice == 1) {
            printf("Enter philosopher number (0 to %d): ", n - 1);
            scanf("%d", &p);
            if (p < 0 || p >= n)
                printf("Invalid philosopher number!\n");
            else

```

```

        philosopher_eat(p);
    }
}

printf("\nSimulation ended.\n");
return 0;
}

```

```

Dining Philosophers Problem
=====
Enter number of philosophers (2-10): 3

Setup: 3 Philosophers, 3 Chopsticks (circular table)
Deadlock rule: At most 2 philosophers can sit at once.

Chopsticks: [C0:FREE] [C1:FREE] [C2:FREE]
Philosophers sitting at table: 0 / 3 (max allowed: 2)

1. Philosopher eats
2. Exit
Choice: 1
Enter philosopher number (0 to 2): 1

--- Philosopher 1 wants to EAT ---
Philosopher 1 sits down. (1 sitting now)
wait(chopstick[1]) ù Philosopher 1 picks up LEFT chopstick.
wait(chopstick[2]) ù Philosopher 1 picks up RIGHT chopstick.
>>> Philosopher 1 is EATING <<<
signal(chopstick[1]) ù Philosopher 1 puts down LEFT chopstick
signal(chopstick[2]) ù Philosopher 1 puts down RIGHT chopstick
Philosopher 1 is now THINKING.

Chopsticks: [C0:FREE] [C1:FREE] [C2:FREE]
Philosophers sitting at table: 0 / 3 (max allowed: 2)

1. Philosopher eats
2. Exit
Choice: |

```

Bankers Algorithm
#include <stdio.h>

```

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

```

```
printf("Enter number of resource types: ");
scanf("%d", &m);
```

```
int allocation[n][m], max[n][m], need[n][m];
int available[m], work[m];
int finish[n], safeSequence[n];
int count = 0, found;
```

```
// Input Allocation Matrix
printf("\nEnter Allocation Matrix:\n");
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        scanf("%d", &allocation[i][j]);
    }
}
```

```
// Input Max Matrix
printf("\nEnter Max Matrix:\n");
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        scanf("%d", &max[i][j]);
    }
}
```

```
// Input Available Resources
printf("\nEnter Available Resources:\n");
for(i = 0; i < m; i++) {
    scanf("%d", &available[i]);
    work[i] = available[i];
}
```

```
// Calculate Need Matrix
for(i = 0; i < n; i++) {
    for(j = 0; j < m; j++) {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}
```

```
// Initialize finish array
for(i = 0; i < n; i++) {
    finish[i] = 0;
}
```

```
// Banker's Algorithm
while(count < n) {
    found = 0;
```

```

for(i = 0; i < n; i++) {
    if(finish[i] == 0) {
        for(j = 0; j < m; j++) {
            if(need[i][j] > work[j]) {
                break;
            }
        }

        if(j == m) {
            for(k = 0; k < m; k++) {
                work[k] += allocation[i][k];
            }

            safeSequence[count] = i;
            finish[i] = 1;
            count++;
            found = 1;
        }
    }
}

if(found == 0) {
    break;
}

// Check Safe State
if(count == n) {
    printf("\nSystem is in SAFE state.\n");
    printf("Safe Sequence: ");
    for(i = 0; i < n; i++) {
        printf("P%d ", safeSequence[i]);
    }
} else {
    printf("\nSystem is in UNSAFE state (Deadlock may occur).\n");
}

return 0;
}

```

```
C:\Users\pc\OneDrive\D x + v
Enter number of processes: 2
Enter number of resource types: 2

Enter Allocation Matrix:
11
22
33
44

Enter Max Matrix:
55
44
66
77

Enter Available Resources:
12
34

System is in UNSAFE state (Deadlock may occur).

Process returned 0 (0x0)  execution time : 27.916 s
Press any key to continue.
|
```

Deadlock detection Algorithm

```
/*
 * Deadlock Detection Algorithm
 * Unit 3 - Chapter 7: Deadlocks
 *
 * FOR SEVERAL INSTANCES OF EACH RESOURCE TYPE
 *
 * DATA STRUCTURES (from notes slide 7.37):
 * Available[j]    = number of available instances of resource j
 * Allocation[i][j] = instances of resource j currently held by process i
 * Request[i][j]   = instances of resource j currently requested by process i
 *
 * ALGORITHM (from notes slides 7.38 - 7.39):
 * -----
 * Step 1: Initialize
 * (a) Work = Available
 * (b) If Allocation[i] != 0 -> Finish[i] = false
 *     If Allocation[i] == 0 -> Finish[i] = true
 *     (process holding nothing is not deadlocked)
```

```

*
* Step 2: Find index i such that BOTH:
*   (a) Finish[i] == false
*   (b) Request[i] <= Work
*   If no such i exists -> go to Step 4
*
* Step 3: Work = Work + Allocation[i]
*   Finish[i] = true
*   Go to Step 2
*
* Step 4: If Finish[i] == false for ANY i -> DEADLOCK
*   Those processes with Finish[i] == false are deadlocked.
*
* NOTES EXAMPLE (slide 7.40):
*   5 processes P0-P4, 3 resource types A(7), B(2), C(6)
*   No deadlock case: sequence <P0,P2,P3,P1,P4> finishes all
*   Deadlock case: P2 requests extra C -> P1,P2,P3,P4 deadlocked
*/

```

```

#include <stdio.h>

```

```

#define MAX_P 10 /* max processes */
#define MAX_R 10 /* max resources */

```

```

int n, m; /* n = processes, m = resource types */
int available[MAX_R];
int allocation[MAX_P][MAX_R];
int request[MAX_P][MAX_R];
int work[MAX_R];
int finish[MAX_P];

```

```

/* Check if Request[i] <= Work */

```

```

int canFinish(int i) {
    int j;
    for (j = 0; j < m; j++)
        if (request[i][j] > work[j])
            return 0;
    return 1;
}

```

```

/* Check if process i holds any resource */

```

```

int hasAllocation(int i) {
    int j;
    for (j = 0; j < m; j++)
        if (allocation[i][j] > 0)
            return 1;
    return 0;
}

```

```

void runDetection() {
    int i, j, found, progress;

    /* STEP 1a: Work = Available */
    for (j = 0; j < m; j++)
        work[j] = available[j];

    /* STEP 1b: Finish[i] = false if Allocation[i] != 0, else true */
    for (i = 0; i < n; i++)
        finish[i] = hasAllocation(i) ? 0 : 1;

    printf("\n--- Step 1: Initialize ---\n");
    printf("Work (= Available): ");
    for (j = 0; j < m; j++) printf("%d ", work[j]);
    printf("\nFinish: ");
    for (i = 0; i < n; i++) printf("P%d=%s ", i, finish[i] ? "T" : "F");
    printf("\n");

    /* STEP 2 & 3: Find a process that can finish */
    printf("\n--- Steps 2 & 3: Find processes that can complete ---\n");
    do {
        found = 0;
        for (i = 0; i < n; i++) {
            /* Step 2: find i where Finish[i]==false AND Request[i]<=Work */
            if (finish[i] == 0 && canFinish(i)) {
                printf("P%d: Request <= Work -> P%d can complete.\n", i, i);

                /* Step 3: Work = Work + Allocation[i], Finish[i] = true */
                for (j = 0; j < m; j++)
                    work[j] += allocation[i][j];
                finish[i] = 1;
                found = 1;

                printf(" Work updated to: ");
                for (j = 0; j < m; j++) printf("%d ", work[j]);
                printf("\n");
            }
        }
    } while (found); /* repeat until no more progress */

    /* STEP 4: Check for deadlock */
    printf("\n--- Step 4: Check Result ---\n");
    printf("Final Finish: ");
    for (i = 0; i < n; i++) printf("P%d=%s ", i, finish[i] ? "T" : "F");
    printf("\n");

    int deadlock = 0;

```

```

printf("\nDeadlocked Processes: ");
for (i = 0; i < n; i++) {
    if (finish[i] == 0) {
        printf("P%d ", i);
        deadlock = 1;
    }
}
if (!deadlock)
    printf("NONE");
printf("\n");

if (deadlock)
    printf("\n*** DEADLOCK DETECTED! ***\n");
else
    printf("\n*** No Deadlock. System is safe. ***\n");
}

int main() {
    int i, j;

    printf("=== Deadlock Detection Algorithm ===\n");
    printf("(Chapter 7 - Several Instances per Resource Type)\n\n");

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);

    /* Available */
    printf("\nEnter Available resources (%d values): ", m);
    for (j = 0; j < m; j++)
        scanf("%d", &available[j]);

    /* Allocation matrix */
    printf("\nEnter Allocation matrix (%d x %d):\n", n, m);
    for (i = 0; i < n; i++) {
        printf(" P%d: ", i);
        for (j = 0; j < m; j++)
            scanf("%d", &allocation[i][j]);
    }

    /* Request matrix */
    printf("\nEnter Request matrix (%d x %d):\n", n, m);
    for (i = 0; i < n; i++) {
        printf(" P%d: ", i);
        for (j = 0; j < m; j++)
            scanf("%d", &request[i][j]);
    }
}

```

```

/* Print the input state */
printf("\n--- Snapshot ---\n");
printf("%-5s %-20s %-20s\n", "Proc", "Allocation", "Request");
for (i = 0; i < n; i++) {
    printf("P%-4d ", i);
    for (j = 0; j < m; j++) printf("%d ", allocation[i][j]);
    printf(" ");
    for (j = 0; j < m; j++) printf("%d ", request[i][j]);
    printf("\n");
}
printf("Available: ");
for (j = 0; j < m; j++) printf("%d ", available[j]);
printf("\n");

/* Run the detection algorithm */
runDetection();

return 0;
}

```

```

Enter number of processes: 3
Enter number of resource types: 3

Enter Available resources (3 values): 11
22
33

Enter Allocation matrix (3 x 3):
P0: 12
13
24
P1: 34
54
32
P2: 21
12
14

Enter Request matrix (3 x 3):
P0: 16
17
19
P1: 11
22
33
P2: 44
55
21

--- Snapshot ---
Proc  Allocation          Request
P0    12 13 24             16 17 19
P1    34 54 32             11 22 33
P2    21 12 14             44 55 21
Available: 11 22 33

--- Step 1: Initialize ---
Work (= Available): 11 22 33
Finish: P0=F P1=F P2=F

--- Steps 2 & 3: Find processes that can complete ---
P1: Request <= Work -> P1 can complete.
    Work updated to: 45 76 65
P2: Request <= Work -> P2 can complete.
    Work updated to: 66 88 79
P0: Request <= Work -> P0 can complete.
    Work updated to: 78 101 103

--- Step 4: Check Result ---
Final Finish: P0=T P1=T P2=T

```

First,Best and worst.c

```
#include <stdio.h>
```

```
int main() {
```

```

int nb, np, i, j;

printf("Enter number of memory blocks: ");
scanf("%d", &nb);

int block[nb], first[nb], best[nb], worst[nb];

printf("Enter sizes of memory blocks: ");
for(i = 0; i < nb; i++) {
    scanf("%d", &block[i]);
    first[i] = block[i];
    best[i] = block[i];
    worst[i] = block[i];
}

printf("Enter number of processes: ");
scanf("%d", &np);

int process[np];
int firstAlloc[np], bestAlloc[np], worstAlloc[np];

printf("Enter sizes of processes: ");
for(i = 0; i < np; i++) {
    scanf("%d", &process[i]);
    firstAlloc[i] = -1;
    bestAlloc[i] = -1;
    worstAlloc[i] = -1;
}

// First Fit
for(i = 0; i < np; i++) {
    for(j = 0; j < nb; j++) {
        if(first[j] >= process[i]) {
            firstAlloc[i] = j;
            first[j] = 0;
            break;
        }
    }
}

// Best Fit
for(i = 0; i < np; i++) {
    int bestIndex = -1;

    for(j = 0; j < nb; j++) {
        if(best[j] >= process[i]) {
            if(bestIndex == -1 || best[j] < best[bestIndex]) {
                bestIndex = j;
            }
        }
    }
}

```

```

    }
}

if(bestIndex != -1) {
    bestAlloc[i] = bestIndex;
    best[bestIndex] = 0;
}
}

// Worst Fit
for(i = 0; i < np; i++) {
    int worstIndex = -1;

    for(j = 0; j < nb; j++) {
        if(worst[j] >= process[i]) {
            if(worstIndex == -1 || worst[j] > worst[worstIndex]) {
                worstIndex = j;
            }
        }
    }

    if(worstIndex != -1) {
        worstAlloc[i] = worstIndex;
        worst[worstIndex] = 0;
    }
}

printf("\nFIRST FIT\n");
for(i = 0; i < np; i++) {
    if(firstAlloc[i] != -1)
        printf("Process %d -> Block %d\n", i + 1, firstAlloc[i] + 1);
    else
        printf("Process %d -> Not Allocated\n", i + 1);
}

printf("\nBEST FIT\n");
for(i = 0; i < np; i++) {
    if(bestAlloc[i] != -1)
        printf("Process %d -> Block %d\n", i + 1, bestAlloc[i] + 1);
    else
        printf("Process %d -> Not Allocated\n", i + 1);
}

printf("\nWORST FIT\n");
for(i = 0; i < np; i++) {
    if(worstAlloc[i] != -1)
        printf("Process %d -> Block %d\n", i + 1, worstAlloc[i] + 1);
}

```

```

        else
            printf("Process %d -> Not Allocated\n", i + 1);
    }

    return 0;
}

```

```

Enter number of memory blocks: 5
Enter sizes of memory blocks: 12
23
13
44
55
Enter number of processes: 3
Enter sizes of processes: 2
21
23

FIRST FIT
Process 1 -> Block 1
Process 2 -> Block 2
Process 3 -> Block 4

BEST FIT
Process 1 -> Block 1
Process 2 -> Block 2
Process 3 -> Block 4

WORST FIT
Process 1 -> Block 5
Process 2 -> Block 4
Process 3 -> Block 2

Process returned 0 (0x0)   execution time : 23.294 s
Press any key to continue.
|

```

Page Replace Algorithm FIFO(first in first out

/*

* Program 7a: FIFO (First-In-First-Out) Page Replacement Algorithm

* Lab Question 7a

*

* RULES (from notes - Chapter 9: Virtual Memory):

```

* -----
* - Reference string evaluated against a fixed number of page frames.
* - Initially all frames are EMPTY.
* - When a page is referenced:
*   -> If the page is already in a frame: NO page fault (HIT).
*   -> If the page is NOT in any frame: PAGE FAULT occurs.
*       * If a free frame is available: load the page into it.
*       * If NO free frame: replace the page that came in FIRST
*         (the oldest page in memory — uses a FIFO queue).
* - FIFO tracks the ORDER in which pages were loaded.
* - The page that entered memory FIRST is the one that gets replaced.
* - How to track: just use a FIFO queue (pointer to the oldest page).
* - Belady's Anomaly: adding more frames can INCREASE page faults in FIFO.
*
* Notes example result: Reference string 7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1
*                       with 3 frames => 15 page faults
*/

```

```

#include <stdio.h>
#include <string.h>

#define MAX_FRAMES 10
#define MAX_PAGES 50

int frames[MAX_FRAMES]; /* current pages in frames */
int frameCount;

/* Check if page is already in memory */
int isInFrame(int page) {
    int i;
    for (i = 0; i < frameCount; i++)
        if (frames[i] == page)
            return 1;
    return 0;
}

/* Display current frame contents */
void displayFrames(int refPage, int fault) {
    int i;
    printf(" %-4d | ", refPage);
    for (i = 0; i < frameCount; i++) {
        if (frames[i] == -1)
            printf(" - ");
        else
            printf(" %d ", frames[i]);
    }
    /* pad remaining */
    for (i = frameCount; i < MAX_FRAMES; i++)

```

```

    printf(" ");
    printf("| %s\n", fault ? "PAGE FAULT" : "");
}

int main() {
    int pages[MAX_PAGES];
    int n, i;
    int pageFaults = 0;
    int fifoPtr = 0; /* points to the OLDEST frame (next one to be replaced) */
    int freeSlots; /* how many frames are still empty */
    printf("Name-Pranjal,USN-1WA24CS212");

    printf("=== FIFO (First-In-First-Out) Page Replacement Algorithm ===\n");
    printf("Rule: Replace the page that came into memory FIRST (oldest page).\n\n");

    printf("Enter number of page frames: ");
    scanf("%d", &frameCount);
    if (frameCount <= 0 || frameCount > MAX_FRAMES) {
        printf("Invalid number of frames! Must be 1 to %d.\n", MAX_FRAMES);
        return 1;
    }

    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);
    if (n <= 0 || n > MAX_PAGES) {
        printf("Invalid number of pages! Must be 1 to %d.\n", MAX_PAGES);
        return 1;
    }

    printf("Enter the reference string (space-separated page numbers): ");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    /* Initialize all frames as empty (-1) */
    for (i = 0; i < frameCount; i++)
        frames[i] = -1;

    freeSlots = frameCount;
    fifoPtr = 0;

    /* Print header */
    printf("\n%-6s | ", "Page");
    for (i = 0; i < frameCount; i++)
        printf("F%d ", i);
    printf("| Status\n");
    printf("-----");
    for (i = 0; i < frameCount; i++) printf("----");
    printf("-----\n");

```

```

/* Process each page reference */
for (i = 0; i < n; i++) {
    int page = pages[i];

    if (isInFrame(page)) {
        /* Page HIT - no fault */
        displayFrames(page, 0);
    } else {
        /* PAGE FAULT */
        pageFaults++;

        if (freeSlots > 0) {
            /* Load into next free frame */
            frames[frameCount - freeSlots] = page;
            freeSlots--;
        } else {
            /* Replace the OLDEST page (FIFO: pointed to by fifoPtr) */
            frames[fifoPtr] = page;
            fifoPtr = (fifoPtr + 1) % frameCount;
        }
        displayFrames(page, 1);
    }
}

printf("-----");
for (i = 0; i < frameCount; i++) printf("----");
printf("-----\n");
printf("\nTotal Page Faults : %d\n", pageFaults);
printf("Total Page Hits   : %d\n", n - pageFaults);
printf("Hit Ratio       : %.2f%%\n", ((float)(n - pageFaults) / n) * 100);

return 0;
}

```

Name-Pranjal,USN-1WA24CS212=== FIFO (First-In-First-Out) Page Replacement Algorithm ===
Rule: Replace the page that came into memory FIRST (oldest page).

Enter number of page frames: 5
Enter number of pages in reference string: 12
Enter the reference string (space-separated page numbers): 11

22
33
44
55
66
77
88
99
01
12
23

Page	F0	F1	F2	F3	F4	Status
11	11	-	-	-	-	PAGE FAULT
22	11	22	-	-	-	PAGE FAULT
33	11	22	33	-	-	PAGE FAULT
44	11	22	33	44	-	PAGE FAULT
55	11	22	33	44	55	PAGE FAULT
66	66	22	33	44	55	PAGE FAULT
77	66	77	33	44	55	PAGE FAULT
88	66	77	88	44	55	PAGE FAULT
99	66	77	88	99	55	PAGE FAULT
1	66	77	88	99	1	PAGE FAULT
12	12	77	88	99	1	PAGE FAULT
23	12	23	88	99	1	PAGE FAULT

Total Page Faults : 12
Total Page Hits : 0
Hit Ratio : 0.00%

Process returned 0 (0x0) execution time : 34.293 s
Press any key to continue.

Page replacement algorithm LRU

/*

* Program 7b: LRU (Least Recently Used) Page Replacement Algorithm

* Lab Question 7b

*

* RULES (from notes - Chapter 9: Virtual Memory):

* -----

* - Use PAST knowledge rather than future.

* - Replace the page that has NOT been used for the MOST amount of time

* (i.e., the least recently used page).

* - Associate TIME OF LAST USE with each page in memory.

* - When a page fault occurs and no free frame is available:

* -> Find the frame whose page was used LEAST RECENTLY (earliest time).

* -> Replace that page.

* - Implementation: Counter method — every time a page is referenced,

* copy a clock/time counter into that page's counter.

* On replacement, find the frame with the SMALLEST counter value.

* - LRU does NOT suffer from Belady's Anomaly (it's a stack algorithm).

* - Generally a good algorithm and frequently used.

*

```

* Notes example result: Reference string 7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1
*
*           with 3 frames => 12 page faults
*           (Better than FIFO=15, worse than OPT=9)
*/

```

```

#include <stdio.h>
#include <string.h>

```

```

#define MAX_FRAMES 10
#define MAX_PAGES 50

```

```

int frames[MAX_FRAMES]; /* current pages in frames */
int lastUsed[MAX_FRAMES]; /* time of last use for each frame (counter) */
int frameCount;

```

```

/* Check if page is in memory; returns frame index or -1 */

```

```

int findInFrame(int page) {
    int i;
    for (i = 0; i < frameCount; i++)
        if (frames[i] == page)
            return i;
    return -1;
}

```

```

/* Find the LRU frame (the one with the smallest lastUsed counter) */

```

```

int findLRUFrame() {
    int i, minTime, minIdx;
    minTime = lastUsed[0];
    minIdx = 0;
    for (i = 1; i < frameCount; i++) {
        if (lastUsed[i] < minTime) {
            minTime = lastUsed[i];
            minIdx = i;
        }
    }
    return minIdx;
}

```

```

/* Find the first free frame (contains -1), or -1 if all full */

```

```

int findFreeFrame() {
    int i;
    for (i = 0; i < frameCount; i++)
        if (frames[i] == -1)
            return i;
    return -1;
}

```

```

void displayFrames(int refPage, int fault) {

```

```

int i;
printf(" %-4d | ", refPage);
for (i = 0; i < frameCount; i++) {
    if (frames[i] == -1)
        printf(" - ");
    else
        printf(" %d ", frames[i]);
}
for (i = frameCount; i < MAX_FRAMES; i++)
    printf(" ");
printf("| %s\n", fault ? "PAGE FAULT" : "");
}

int main() {
    int pages[MAX_PAGES];
    int n, i;
    int pageFaults = 0;
    int timer = 0;
    printf("Name-Pranjal,USN-1WA24CS212");
    /* global clock/counter to track usage time */

    printf("=== LRU (Least Recently Used) Page Replacement Algorithm ===\n");
    printf("Rule: Replace the page that has NOT been used for the MOST amount of time.\n");
    printf("    (Uses counter: each page's counter = time of its last reference)\n\n");

    printf("Enter number of page frames: ");
    scanf("%d", &frameCount);
    if (frameCount <= 0 || frameCount > MAX_FRAMES) {
        printf("Invalid number of frames! Must be 1 to %d.\n", MAX_FRAMES);
        return 1;
    }

    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);
    if (n <= 0 || n > MAX_PAGES) {
        printf("Invalid number of pages! Must be 1 to %d.\n", MAX_PAGES);
        return 1;
    }

    printf("Enter the reference string (space-separated page numbers): ");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    /* Initialize all frames as empty */
    for (i = 0; i < frameCount; i++) {
        frames[i] = -1;
        lastUsed[i] = 0;
    }
}

```

```

/* Print header */
printf("\n%-6s | ", "Page");
for (i = 0; i < frameCount; i++)
    printf("F%d ", i);
printf("| Status\n");
printf("-----");
for (i = 0; i < frameCount; i++) printf("----");
printf("-----\n");

/* Process each page reference */
for (i = 0; i < n; i++) {
    int page = pages[i];
    int idx;
    timer++; /* increment global clock each reference */

    idx = findInFrame(page);

    if (idx != -1) {
        /* Page HIT - update its last-used time */
        lastUsed[idx] = timer;
        displayFrames(page, 0);
    } else {
        /* PAGE FAULT */
        pageFaults++;

        int freeldx = findFreeFrame();
        if (freeldx != -1) {
            /* Load into free frame */
            frames[freeldx] = page;
            lastUsed[freeldx] = timer;
        } else {
            /* Replace LRU page: find frame with smallest lastUsed */
            int lruIdx = findLRUFrame();
            frames[lruIdx] = page;
            lastUsed[lruIdx] = timer;
        }
        displayFrames(page, 1);
    }
}

printf("-----");
for (i = 0; i < frameCount; i++) printf("----");
printf("-----\n");
printf("\nTotal Page Faults : %d\n", pageFaults);
printf("Total Page Hits   : %d\n", n - pageFaults);
printf("Hit Ratio       : %.2f%%\n", ((float)(n - pageFaults) / n) * 100);

```

```

return 0;
}

```

```

Name-Pranjal,USN-1WA24CS212=== FIFO (First-In-First-Out) Page Replacement Algorithm ===
Rule: Replace the page that came into memory FIRST (oldest page).

Enter number of page frames: 5
Enter number of pages in reference string: 12
Enter the reference string (space-separated page numbers): 11
22
33
44
55
66
77
88
99
01
12
23

Page | F0 F1 F2 F3 F4 | Status
-----
11 | 11 - - - - | PAGE FAULT
22 | 11 22 - - - | PAGE FAULT
33 | 11 22 33 - - | PAGE FAULT
44 | 11 22 33 44 - | PAGE FAULT
55 | 11 22 33 44 55 | PAGE FAULT
66 | 66 22 33 44 55 | PAGE FAULT
77 | 66 77 33 44 55 | PAGE FAULT
88 | 66 77 88 44 55 | PAGE FAULT
99 | 66 77 88 99 55 | PAGE FAULT
1 | 66 77 88 99 1 | PAGE FAULT
12 | 12 77 88 99 1 | PAGE FAULT
23 | 12 23 88 99 1 | PAGE FAULT
-----

Total Page Faults : 12
Total Page Hits : 0
Hit Ratio : 0.00%

Process returned 0 (0x0) execution time : 34.293 s
Press any key to continue.
|

```

Page replacement Algorithm

Optimal

/*

* Program 7c: Optimal (OPT) Page Replacement Algorithm

* Lab Question 7c

*

* RULES (from notes - Chapter 9: Virtual Memory):

* -----

* - Replace the page that will NOT be used for the LONGEST period of time in the FUTURE.

* - This is a THEORETICAL algorithm — cannot be implemented in practice because it requires future knowledge of the reference string.

* - Used as a BENCHMARK to measure how well other algorithms perform.

* - From the notes: "9 is optimal for the example"

* (Reference string 7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

* with 3 frames => 9 page faults — the MINIMUM possible)

```

*
* ALGORITHM STEPS:
* When a page fault occurs and no free frame is available:
* For each page currently in a frame:
*   -> Look FORWARD in the reference string from current position.
*   -> Find the NEXT time that page will be used.
*   -> If a page will NEVER be used again, replace it immediately.
*   -> Otherwise, replace the page whose next use is FARTHEST in the future.
*/

```

```

#include <stdio.h>
#include <string.h>

```

```

#define MAX_FRAMES 10
#define MAX_PAGES 50

```

```

int frames[MAX_FRAMES];
int frameCount;

```

```

/* Check if page is in memory; returns frame index or -1 */
int findInFrame(int page) {
    int i;
    for (i = 0; i < frameCount; i++)
        if (frames[i] == page)
            return i;
    return -1;
}

```

```

/* Find the first free frame, returns index or -1 */
int findFreeFrame() {
    int i;
    for (i = 0; i < frameCount; i++)
        if (frames[i] == -1)
            return i;
    return -1;
}

```

```

/*
* Find which frame to replace using Optimal policy.
* Look ahead in the reference string from position 'currentPos'.
* Replace the page whose NEXT use is farthest in the future.
* If a page will never be used again, replace it immediately.
*/
int findOptimalVictim(int *pages, int n, int currentPos) {
    int i, j;
    int farthest = -1; /* farthest next-use position found so far */
    int victimFrame = 0; /* frame index to replace */

```

```

for (i = 0; i < frameCount; i++) {
    int nextUse = n; /* assume page will never be used again (beyond end) */

    /* Search forward for next use of frames[i] */
    for (j = currentPos; j < n; j++) {
        if (pages[j] == frames[i]) {
            nextUse = j;
            break;
        }
    }

    /* If this page will never be used again, replace it immediately */
    if (nextUse == n) {
        return i;
    }

    /* Otherwise, keep track of the farthest next-use */
    if (nextUse > farthest) {
        farthest = nextUse;
        victimFrame = i;
    }
}
return victimFrame;
}

void displayFrames(int refPage, int fault) {
    int i;
    printf(" %-4d | ", refPage);
    for (i = 0; i < frameCount; i++) {
        if (frames[i] == -1)
            printf(" - ");
        else
            printf(" %d ", frames[i]);
    }
    for (i = frameCount; i < MAX_FRAMES; i++)
        printf(" ");
    printf("| %s\n", fault ? "PAGE FAULT" : "");
}

int main() {
    int pages[MAX_PAGES];
    int n, i;
    int pageFaults = 0;
    printf("Name-Pranjal,USN-1WA24CS212");

    printf("=== Optimal (OPT) Page Replacement Algorithm ===\n");
    printf("Rule: Replace the page that will NOT be used for the LONGEST time in future.\n");

```

```
printf("    (Theoretical algorithm — requires future knowledge of reference string)\n");
printf("    Used as a benchmark: gives the MINIMUM possible page faults.\n\n");
```

```
printf("Enter number of page frames: ");
scanf("%d", &frameCount);
if (frameCount <= 0 || frameCount > MAX_FRAMES) {
    printf("Invalid number of frames! Must be 1 to %d.\n", MAX_FRAMES);
    return 1;
}
```

```
printf("Enter number of pages in reference string: ");
scanf("%d", &n);
if (n <= 0 || n > MAX_PAGES) {
    printf("Invalid number of pages! Must be 1 to %d.\n", MAX_PAGES);
    return 1;
}
```

```
printf("Enter the reference string (space-separated page numbers): ");
for (i = 0; i < n; i++)
    scanf("%d", &pages[i]);
```

```
/* Initialize all frames as empty */
for (i = 0; i < frameCount; i++)
    frames[i] = -1;
```

```
/* Print header */
printf("\n%-6s | ", "Page");
for (i = 0; i < frameCount; i++)
    printf("F%d ", i);
printf("| Status\n");
printf("-----");
for (i = 0; i < frameCount; i++) printf("----");
printf("-----\n");
```

```
/* Process each page reference */
for (i = 0; i < n; i++) {
    int page = pages[i];

    if (findInFrame(page) != -1) {
        /* Page HIT */
        displayFrames(page, 0);
    } else {
        /* PAGE FAULT */
        pageFaults++;

        int freeldx = findFreeFrame();
        if (freeldx != -1) {
            /* Load into free frame */
```

```

        frames[freeldx] = page;
    } else {
        /* Optimal replacement: look ahead from NEXT position (i+1) */
        int victimFrame = findOptimalVictim(pages, n, i + 1);
        frames[victimFrame] = page;
    }
    displayFrames(page, 1);
}
}

printf("-----");
for (i = 0; i < frameCount; i++) printf("----");
printf("-----\n");
printf("\nTotal Page Faults : %d\n", pageFaults);
printf("Total Page Hits   : %d\n", n - pageFaults);
printf("Hit Ratio       : %.2f%%\n", ((float)(n - pageFaults) / n) * 100);

return 0;
}

```

Name-Pranjal,USN-1WA24CS212=== FIFO (First-In-First-Out) Page Replacement Algorithm ===
Rule: Replace the page that came into memory FIRST (oldest page).

Enter number of page frames: 5
Enter number of pages in reference string: 12
Enter the reference string (space-separated page numbers): 11

22
33
44
55
66
77
88
99
01
12
23

Page	F0	F1	F2	F3	F4	Status
------	----	----	----	----	----	--------

11	11	-	-	-	-	PAGE FAULT
22	11	22	-	-	-	PAGE FAULT
33	11	22	33	-	-	PAGE FAULT
44	11	22	33	44	-	PAGE FAULT
55	11	22	33	44	55	PAGE FAULT
66	66	22	33	44	55	PAGE FAULT
77	66	77	33	44	55	PAGE FAULT
88	66	77	88	44	55	PAGE FAULT
99	66	77	88	99	55	PAGE FAULT
1	66	77	88	99	1	PAGE FAULT
12	12	77	88	99	1	PAGE FAULT
23	12	23	88	99	1	PAGE FAULT

Total Page Faults : 12
Total Page Hits : 0
Hit Ratio : 0.00%

Process returned 0 (0x0) execution time : 34.293 s
Press any key to continue.